Project II

Blanca Loma Mecha
Yujie Zhang

Embedded System with Raspberry PI

INDEX

1. Develop an embedded Linux with Buildroot

1.1. Buildroot Configuration

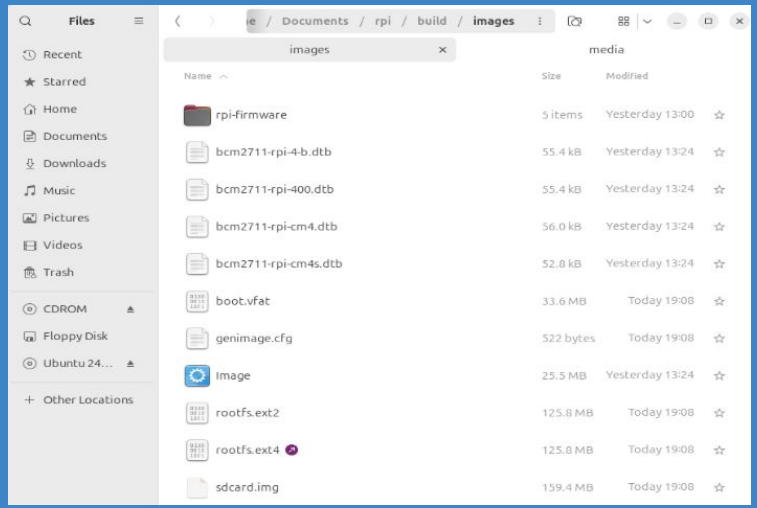
1.2. Boot and Connection

1.3. Cross-Compilation

PDF → Design of Embedded Systems with Raspberry PI v1.0

1.1 Buildroot Configuration

Configure Buildroot for RPI4 with basic configuration



Perform make to create the image

Format our microSD card

Copy the sdcard.img file

```
sudo dmesg
sudo dd if=./images/sdcard.img of=/dev/sdb bs=10M
sudo sync
```

We also changed config.txt and cmdline.txt files.

1.2 Boot and connection

RPI connector	FDTI Terminal
RXD UART (GPIO16) Pin 10	Terminal 4 (Orange)
TXD UART (GPIO15) Pin 8	Terminal 5 (Yellow)
Ground (GND) Pin 6	Pin 1

-Introduce the SD card

-Connect the FTDI terminal and the power supply

-Found the device type:

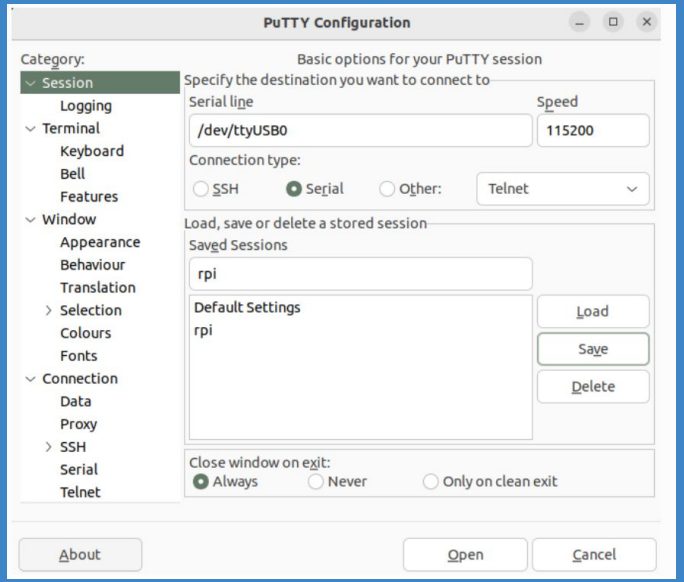
```
sudo dmesg | grep tty
```

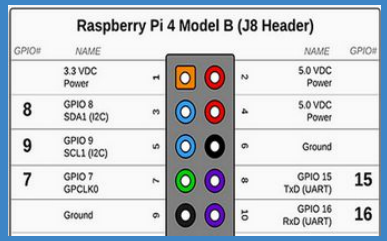
```
ttyUSB0
```

-Execute udo putty:

-Enter credentials:

```
buildroot login: root
Password:
#
```





1.1 Buildroot Configuration

```
Target options  --->
Toolchain  --->
Build options  --->
System configuration  --->
Kernel  --->
Target packages  --->
Filesystem images  --->
Bootloaders  --->
Host utilities  --->
Legacy config options  --->
```

A) Add Wi-fi support to connect via SSH.

Added the Broadcom firmware support for Wireless hardware

Added some commands in the post-build.sh script

```
cp <buildroot-folder>/package/busybox/S10mdev ${TARGET_DIR}/etc/init.d/S10mdev
chmod 755 ${TARGET_DIR}/etc/init.d/S10mdev
cp <buildroot-folder>/package/busybox/mdev.conf ${TARGET_DIR}/etc/mdev.conf
```

```
cp <buildroot-folder>/board/raspberrypi4-64/interfaces ${TARGET_DIR}/etc/network/interfaces
cp <buildroot-folder>/board/raspberrypi4-64/wpa_supplicant.conf ${TARGET_DIR}/etc/wpa_supplicant.conf
```

Created the files interfaces and wpa_supplicant.conf where we can define as many WIFIs as we want.

1.1 Buildroot Configuration

B) Customize the Linux kernel to configure I2C.

We tried to detect the I2C device and it wasn't present

```
# ls /dev/i2c*
```

We activated:

-make … menuconfig → i2c tools

-make … linux-menuconfig → I2C device interface

I2C Hardware Bus support

Broadcom BCM2835 I2C controller

BCM2708 BSC

1.2 Boot and connection

-After creating and copying the image => boot RPI

-Run sudo putty and enter credentials

-SSH:

Find IP => `inet addr:192.168.1.133`

Allow root login => `# vi /etc/ssh/sshd_config`

`PermitRootLogin yes`

Enter as root =>

```
alumno@m31:~/Escritorio$ ssh root@192.168.1.133
root@192.168.1.133's password:
#
```

-

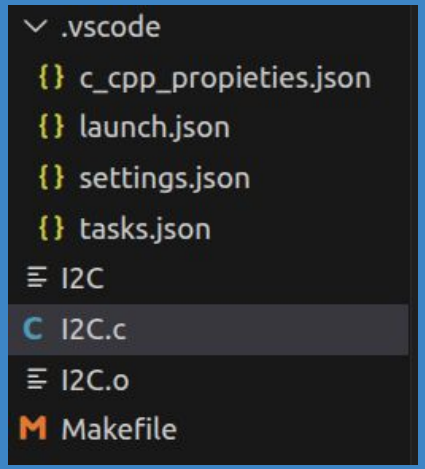
1.3 Cross-compilation

We reuse the I2C.c file from Project I

Follow 6.3 point of the PDF => VSCode

-Install C/C++ extension

```
# cd /root
# ./I2C
```




```
{
    "buildroot_build_dir": "/home/alumno/Documentos/rpi/build",
    "target_ip": "192.168.1.133",
    "executable_file": "I2C",
    "buildroot_env": "${config:buildroot_build_dir}/host/environment-setup",
    "host_gdb": "${config:buildroot_build_dir}/host/bin/aarch64-linux-gdb",
}
```

1.3 Cross-compilation

- Makefile => It **automates the compilation** process

- settings.json => defines the **environment and paths** needed for cross-compilation.

- tasks.json => defines the **commands** to **build, deploy** and **debug** the application.

- launch.json => defines the **remote debugging** configuration

- c_cpp_properties.json => allows VSCode to **find** the correct **headers and libraries** for the target system

2. HOW TO DEVELOP CODE FOR COLOR SENSOR IN PROJECT II

2.1 Code developed in Project I for the accelerometer

2.2 Previous action before modificating code of project II

2.3 Main steps of our implementation of Project II

2.1 Code developed in Project I for the accelerometer

```
#define w_len 1
#define r_len 6
```

- Writing length and reading length

```
wake[0] = 0x6B;
wake[1] = 0x00;

messages[0].addr   = addr;
messages[0].flags  = 0; // 0 = write
messages[0].len    = 2;
messages[0].buf    = wake;

packets.msgs = messages;
packets.nmsgs = 1;
ioctl(fd, I2C_RDWR, &packets);
```

- Register 0x68 was used to wake up the accelerometer

- Register 0x1C controlled its sensitivity

```
write_bytes[0] = 0x1C;

messages[0].addr   = addr;
messages[0].flags  = 0; // 0 = write
messages[0].len    = w_len;
messages[0].buf    = write_bytes;

messages[1].addr   = addr;
messages[1].flags  = I2C_M_RD;  // read
messages[1].len    = 1;
messages[1].buf    = sensib;

packets.msgs = messages;
packets.nmsgs = 2;
ioctl(fd, I2C_RDWR, &packets);

int8_t escala = sensib[0];
printf("Escala: %d\n", escala);
```

2.1 Code developed in Project I for the accelerometer

```
write_bytes[0] = 0x3B; //to read X[15:8]

while(1)
{

  messages[0].addr   = addr;
  messages[0].flags  = 0; // 0 = write
  messages[0].len    = w_len;
  messages[0].buf    = write_bytes;

  messages[1].addr   = addr;
  messages[1].flags  = I2C_M_RD;  // read
  messages[1].len    = r_len;
  messages[1].buf    = read_bytes;

  packets.msgs = messages;
  packets.nmsgs = 2;
  ioctl(fd, I2C_RDWR, &packets);
```

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT[7:0]							

- Continuously read data starting from register 0x3B for X, Y and Z data

```
float x = (float)((int16_t)(read_bytes[0] << 8 | read_bytes[1]) / sensibilidad);
float y = (float)((int16_t)(read_bytes[2] << 8 | read_bytes[3]) / sensibilidad);
float z = (float)((int16_t)(read_bytes[4] << 8 | read_bytes[5]) / sensibilidad);

printf("X: %.3f, Y: %.3f, Z: %.3f\n", x, y, z);
sleep(1);
```

2.2 Previous action before modificating code of project II

➢ Address -> Command “i2cdetect”

```
# i2cdetect 1
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will probe file /dev/i2c-1.
I will probe address range 0x08-0x77.
Continue? [Y/n] Y
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- 29 -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

➢ Command bit: 0x80

```
#define CMD_BIT 0x80
```

2.3 Main steps of our implementation of Project II - Step I

page.13 datasheet TC34725

```
write_bytes[0] = 0x12 | CMD_BIT;

messages[0].addr   = addr_col;
messages[0].flags  = 0;
messages[0].len    = w_len;
messages[0].buf    = write_bytes;
```

```
messages[1].addr   = addr_col;
messages[1].flags  = I2C_M_RD;
messages[1].len    = 1;
messages[1].buf    = id_reg;

packets.msgs = messages;
packets.nmsgs = 2;
ioctl(fd, I2C_RDWR, &packets);

printf("Sensor ID: 0x%02X\n", (unsigned char)id_reg[0]);
```

- Send the address of the **ID** register, 0x12, and then read the returned value

2.3 Main steps of our implementation of project II - Step II

Enable Register (0x00)

	7	6	5	4	3	2	1	0	
	0	0	0	0	0	0	1	1	
ENABLE	Reserved			AIEN	WEN	Reserved	AEN	PON	Address 0x00

page.15 datasheet TC34725

```
enable[0] = 0x00 | CMD_BIT;//register
enable[1] = 0x03;//value we want to put in the register

messages[0].addr   = addr_col;
messages[0].flags  = 0;
messages[0].len    = 2;
messages[0].buf    = enable;

packets.msgs = messages;
packets.nmsgs = 1;
ioctl(fd, I2C_RDWR, &packets);
```

- PON powers on the internal oscillator, and AEN enables the ADC channels

2.3 Main Steps of our Implementation of Project II - Step III

RGBC Timing Register (0x01)

FIELD	BITS	DESCRIPTION			
		VALUE	INTEG_CYCLES	TIME	MAX COUNT
ATIME	7:0	0xFF	1	2.4 ms	1024
		0xF6	10	24 ms	10240
		0xD5	42	101 ms	43008

page.16 datasheet TC34725

```
atime[0] = 0x01;
atime[1] = 0xD5;

messages[0].addr  = addr;
messages[0].flags = 0;
messages[0].len   = 2;
messages[0].buf   = atime;

packets.msgs = messages;
packets.nmsgs = 1;
ioctl(fd, I2C_RDWR, &packets);
```

- Configure the integration time using the ATIME register at address 0x01 with value 0xD5, which corresponds to about 101 milliseconds

2.3 MAIN STEPS OF OUR IMPLEMENTATION OF PROJECT II - STEP IV

REGISTER	ADDRESS	BITS	DESCRIPTION
CDATA	0x14	7:0	Clear data low byte
CDATAH	0x15	7:0	Clear data high byte
RDATA	0x16	7:0	Red data low byte
RDATAH	0x17	7:0	Red data high byte
GDATA	0x18	7:0	Green data low byte
GDATAH	0x19	7:0	Green data high byte
BDATA	0x1A	7:0	Blue data low byte
BDATAH	0x1B	7:0	Blue data high byte

The data is stored in consecutive registers starting from 0x14, with each channel using two bytes.

page.19 datasheet TC34725

```
#define w_len 1
#define r_len 8
```

We read a total of 8 bytes, then combine each pair into 16-bit values representing the light intensity of each channel.

2.3 Main steps of our implementation of project II - Step IV

```
write_bytes[0] = 0x14;

while(1)
{
  // Message WRITE: send the registre that we want to read
  messages[0].addr   = addr;
  messages[0].flags  = 0; // 0 = write
  messages[0].len    = w_len;
  messages[0].buf    = write_bytes;

  // Message READ: read 8 bytes from the sensor
  messages[1].addr   = addr;
  messages[1].flags  = I2C_M_RD;
  messages[1].len    = r_len;
  messages[1].buf    = read_bytes;

  // Construct and send packets
  packets.msgs = messages;
  packets.nmsgs = 2;
  ioctl(fd, I2C_RDWR, &packets);
```

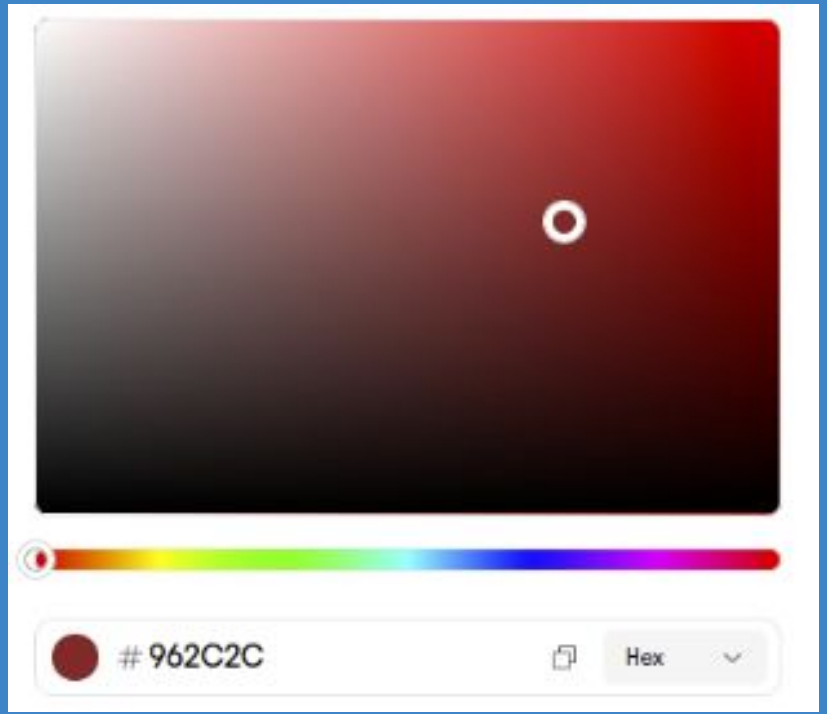
```
uint16_t clear = (uint16_t)((read_bytes_col[1] << 8) | (unsigned char)read_bytes_col[0]);
uint16_t red   = (uint16_t)((read_bytes_col[3] << 8) | (unsigned char)read_bytes_col[2]);
uint16_t green = (uint16_t)((read_bytes_col[5] << 8) | (unsigned char)read_bytes_col[4]);
uint16_t blue  = (uint16_t)((read_bytes_col[7] << 8) | (unsigned char)read_bytes_col[6]);

printf("COLOUR SENSOR:  C: %u, R: %u, G: %u, B: %u\n", clear, red, green, blue);
```

These values are then printed on the screen in real time and can be used to control the RGB LED backlight.

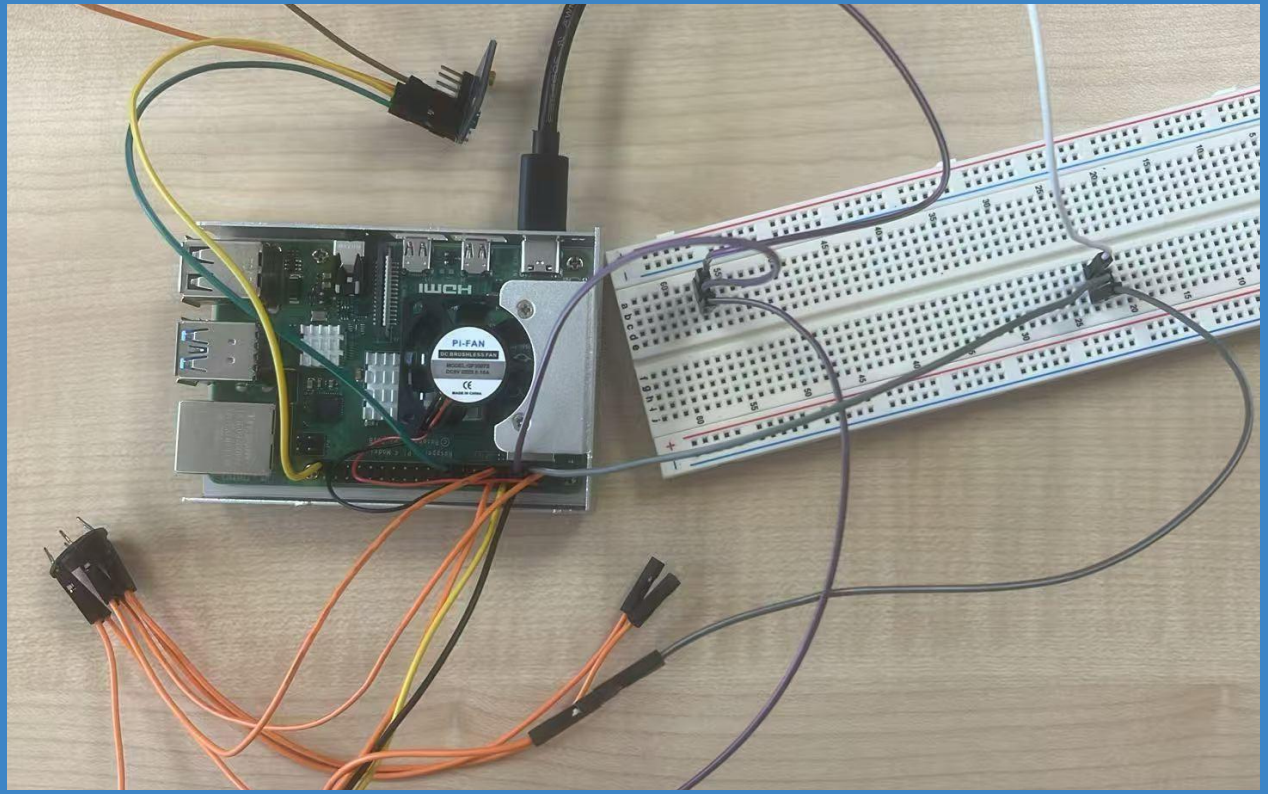
3. RESULT OF THE COLOR SENSOR IN PROJECT II

```
# ./I2C
Sensor ID: 0x44
C: 13320, R: 8345, G: 2351, B: 2704
RGB 8-bit -> R: 159, G: 45, B: 51
C: 13211, R: 8289, G: 2330, B: 2671
RGB 8-bit -> R: 159, G: 44, B: 51
C: 13453, R: 8333, G: 2429, B: 2775
RGB 8-bit -> R: 157, G: 46, B: 52
C: 13721, R: 8579, G: 2422, B: 2768
RGB 8-bit -> R: 159, G: 45, B: 51
C: 13715, R: 8556, G: 2439, B: 2797
RGB 8-bit -> R: 159, G: 45, B: 52
^Z[13]+  Stopped                 ./
#
```

Hex #962C2C RGB 150, 44, 44 HSL 0, 55, 38 OKLCH 0.46, 0.14, 25



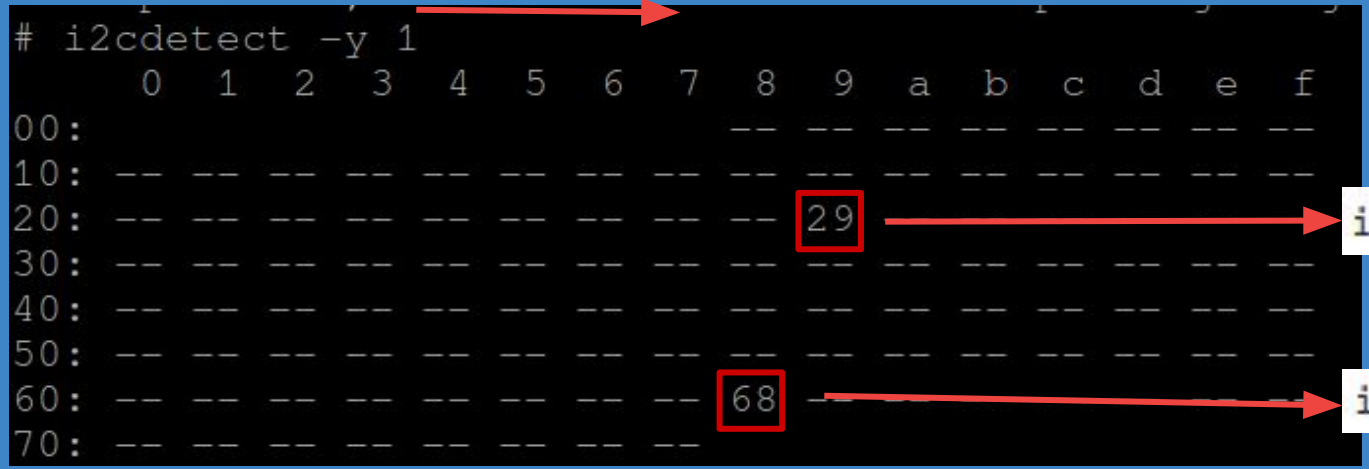
4. Final combination of both codes



For connecting the accelerometer and the color sensor together, use a breadboard to avoid problems in the connections and possible damage in the pins

4. Final combination of both codes

- Address -> Command “i2cdetect”




```
int addr_col = 0x29;
```

```
int addr_acc = 0x68;
```

- Address of the accelerometer: 0x29
- Address of the color senor: ox68

4. Final combination of both codes

```
#define w_len 1
#define r_len_col 8
#define r_len_acc 6
#define CMD_BIT 0x80
#define MAX_COUNT 44032
#define sensibilidad 16384.0
struct i2c_rdwr_ioctl_data packets;
struct i2c_msg messages[2];
char write_bytes[w_len];
char wake[2];
char sensib[1];
char enable[2];
char atime[2];
char control[2];
char id_reg[1];
char read_bytes_acc[r_len_acc];
char read_bytes_col[r_len_col];
```

Reading length for the accelerometer

Reading length for the color sensor

Parameters for the accelerometer

Parameters for the accelerometer

Array of reading data for the accelerometer

Array of reading data for the color sensor

4. Final combination of both codes - RESULTS

```
# ./I2C
Escala: 0
Sensor ID: 0x44
ACCELEROMETER:  X: 0.168, Y: 0.213, Z: -1.096
COLOUR SENSOR:  C: 16208, R: 9349, G: 3237, B: 3492
ACCELEROMETER:  X: 0.166, Y: 0.219, Z: -1.102
COLOUR SENSOR:  C: 16143, R: 9305, G: 3229, B: 3473
ACCELEROMETER:  X: 0.267, Y: -0.902, Z: 0.296
COLOUR SENSOR:  C: 7688, R: 3598, G: 2020, B: 1838
ACCELEROMETER:  X: 0.045, Y: -0.579, Z: 0.444
COLOUR SENSOR:  C: 7687, R: 3598, G: 2019, B: 1837
ACCELEROMETER:  X: -0.627, Y: -1.141, Z: -0.155
COLOUR SENSOR:  C: 5786, R: 1877, G: 1948, B: 1585
ACCELEROMETER:  X: -0.443, Y: -0.900, Z: -0.135
COLOUR SENSOR:  C: 5711, R: 1841, G: 1929, B: 1567
ACCELEROMETER:  X: -0.536, Y: 0.763, Z: -0.051
COLOUR SENSOR:  C: 5717, R: 1843, G: 1931, B: 1569
ACCELEROMETER:  X: -0.971, Y: 0.566, Z: -0.185
COLOUR SENSOR:  C: 5725, R: 1846, G: 1934, B: 1571
ACCELEROMETER:  X: -0.913, Y: -0.232, Z: -0.159
COLOUR SENSOR:  C: 5724, R: 1846, G: 1933, B: 1570
```

Results for the accelerometer in range [-2,-2] corresponds to sensitivity 16384

Results for the color sensor in range [0, 44032] corresponds to the integration time 101ms

Thank you!!!